

Bacharelado em Sistemas de Informação

Disciplina: Organização de Computadores

Tema: Representação de Dados

Prof: Loham Silva

Aula 03: Representação de Dados

- Tipos de Dados;
- Tipo Caractere
- Tipo Lógico
- Tipo Numérico
- Sinal e Magnitude;
- Ponto Fixo
- Ponto Flutuante
- Overflow e Underflow



Introdução

Quando um programador escreve um programa, é necessário definir qual é o formato de um dado e como esse dado será manipulado. Exemplo:

VAR ANOS: INTEGER

Como o computador só entende a numeração binária, há sempre a necessidade de conversão. Ao ler a palavra “INTEGER” por exemplo, o sistema o interpreta, separa memória disponível para o dado, organiza os bits que representa o número, dentre outras operações.

Para saber o que fazer, um computador lida com tipos primitivos de dados, implementados por linguagens de programação.

Tipos de Dados

O tipo primitivo de dados é caracterizado por: **CARACTERE** (onde contempla símbolos alfanuméricos), **BOOLEANO** (para dados do tipo lógico) e **NUMÉRICO** (representam números inteiros e fracionários).

FORMA	TIPOS
Caractere	Caractere
Lógico	Lógico
Numérico	Sinal e magnitude Complemento a 1 Complemento a 2
Ponto fixo Ponto flutuante Decimal	

Tipos de Dados

Um tipo primitivo de dados é a representação computacional de algum conceito utilizado no nosso cotidiano: números, letras, símbolos, um valor verdadeiro ou falso.

Já os tipos não primitivos de dados são estruturas mais complexas e podem ser:

- vetores
- matrizes
- classes
- objetos
- enumerações
- dentre outros..

Tipos de Dados

Tipo	Em bit	Bits	Bytes																																																																
char	8	<table><tr><td>0</td><td>0</td><td>0</td><td>0</td><td>1</td><td>1</td><td>1</td><td>1</td></tr></table>	0	0	0	0	1	1	1	1	8 bits = 1 byte																																																								
0	0	0	0	1	1	1	1																																																												
int	16	<table><tr><td>0</td><td>0</td><td>0</td><td>0</td><td>1</td><td>1</td><td>1</td><td>1</td></tr><tr><td>1</td><td>1</td><td>1</td><td>1</td><td>0</td><td>0</td><td>0</td><td>0</td></tr></table>	0	0	0	0	1	1	1	1	1	1	1	1	0	0	0	0	16 bits = 2 bytes																																																
0	0	0	0	1	1	1	1																																																												
1	1	1	1	0	0	0	0																																																												
float	32	<table><tr><td>0</td><td>0</td><td>0</td><td>0</td><td>1</td><td>1</td><td>1</td><td>1</td></tr><tr><td>1</td><td>1</td><td>1</td><td>1</td><td>0</td><td>0</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>1</td><td>1</td><td>1</td><td>1</td><td>1</td><td>0</td></tr><tr><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td></tr></table>	0	0	0	0	1	1	1	1	1	1	1	1	0	0	0	0	1	1	1	1	1	1	1	0	0	0	0	0	0	0	0	0	32 bits = 4 bytes																																
0	0	0	0	1	1	1	1																																																												
1	1	1	1	0	0	0	0																																																												
1	1	1	1	1	1	1	0																																																												
0	0	0	0	0	0	0	0																																																												
double	64	<table><tr><td>0</td><td>0</td><td>0</td><td>0</td><td>1</td><td>1</td><td>1</td><td>1</td></tr><tr><td>1</td><td>1</td><td>1</td><td>1</td><td>0</td><td>0</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>1</td><td>1</td><td>1</td><td>1</td><td>1</td><td>0</td></tr><tr><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>0</td><td>0</td><td>0</td><td>1</td><td>1</td><td>1</td><td>1</td></tr><tr><td>1</td><td>1</td><td>1</td><td>1</td><td>0</td><td>0</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>1</td><td>1</td><td>1</td><td>1</td><td>1</td><td>0</td></tr><tr><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td></tr></table>	0	0	0	0	1	1	1	1	1	1	1	1	0	0	0	0	1	1	1	1	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1	0	0	0	0	1	1	1	1	1	1	1	0	0	0	0	0	0	0	0	0	64 bits = 8 bytes
0	0	0	0	1	1	1	1																																																												
1	1	1	1	0	0	0	0																																																												
1	1	1	1	1	1	1	0																																																												
0	0	0	0	0	0	0	0																																																												
0	0	0	0	1	1	1	1																																																												
1	1	1	1	0	0	0	0																																																												
1	1	1	1	1	1	1	0																																																												
0	0	0	0	0	0	0	0																																																												
void	0	0	Nada																																																																

Tipos de dados e espaço de memória disponível.

Quando uma variável é declarada em um programa, um espaço de memória é alocado para guardar o dado pertencente à variável.

Representação de dados

Exercício



- 1) Qual a diferença entre dados primitivos e não primitivos?**
- 2) Como você acha que um computador entende um array?**
- 3) Quantos bytes em memória ocupam:**
 - a) um inteiro
 - b) um float
 - c) um double
 - d) um char
 - e) um booleano
- 4) O que o sistema faz quando se depara com a instrução: “int idade;”?**

Tipos Caractere

Para o tipo caractere, o computador necessita realizar uma especificação de correspondência entre o símbolo da informação e um grupo de algarismos binários. Para isso, utiliza os padrões:

BCD - Binary Coded Decimal: realiza a correspondência de 6 bits por caractere, permitindo então a codificação de 64 símbolos diferentes. ($2^6 = 64$)

EBCDIC - Extended Binary Coded Decimal: Exclusivo da IBM para computadores de grande porte, realiza a correspondência de 8 bits por caractere, permitindo então a codificação de 256 símbolos diferentes. ($2^8 = 256$)

ASCII - American Standard Code for Information Interchange: realiza a correspondência de 7 bits por caractere, permitindo então a codificação de 128 símbolos diferentes. ($2^7 = 128$)

UNICODE - realiza a correspondência de 16 bits por caractere, permitindo a codificação de 65536 símbolos diferentes. ($2^{16}=65536$)

Tabela ASCII

<i>Bits</i> 3210	654 <i>000</i>	<i>001</i>	<i>010</i>	<i>011</i>	<i>100</i>	<i>101</i>	<i>110</i>	<i>111</i>
0000	NUL	DLE	SP	0	@	P	\	p
0001	SOH	DC1	!	1	A	Q	a	q
0010	STX	DC2	"	2	B	R	b	r
0011	ETX	DC3	#	3	C	S	c	s
0100	EOT	DC4	\$	4	D	T	d	t
0101	ENQ	NAK	%	5	E	U	e	u
0110	ACK	SYN	&	6	F	V	f	v
0111	BEL	ETB	'	7	G	W	g	w
1000	BS	CAN	(	8	H	X	h	x
1001	HT	EM	)	9	I	Y	i	y
1010	LF	SUB	*	:	J	Z	j	z
1011	VT	ESC	+	;	K	[	k	{
1100	FF	FS	,	<	L	\	l	
1101	CR	GS	-	=	M	]	m	}
1110	SO	RS	.	>	N	^	n	~
1111	SI	US	/	?	O	_	o	DEL

Tabela ASCII

A letra Q é representada por:

6	5	4	3	2	1	0
1	0	1	0	0	0	1

Tabela EBCDIC (IBM)

Bits	7654															
3210	0000	0001	0010	0011	0100	0101	0110	0111	1000	1001	1010	1011	1100	1101	1110	1111
0000	NUL	DLE	DS		SP	&	-						[	]	\	φ
0001	SOH	DC1	SOS						a	j	-		A	J		1
0010	STX	DC2	FS	SYN					b	k	s		B	K	S	2
0011	ETX	DC3							c	l	t		C	L	T	3
0100	PF	RES	BYP	PN					d	m	u		D	M	U	4
0101	HT	NL	LF	RS					e	n	v		E	N	V	5
0110	LC	BS	EOB	UC					f	o	w		F	O	W	6
0111	DEL	IL	ESC	EOT					g	p	x		G	P	X	7
1000		CAN							h	q	y		H	Q	Y	8
1001	RLF	EM						\	i	r	z		I	R	Z	9
1010	SMM	CC	SM		α	!	!	:								
1011	VT				'	\$	'	#								
1100	FF	IFS		DC4	<	*	%	@								
1101	CR	IGS	ENQ	NAK	(	)	-	'								
1110	SO	IRS	ACK		+	:	>	=								
1111	SI	IUS	BEL	SUB		~	?	"								

Representação de caracteres

O programa abaixo escrito em Basic foi convertido para a linguagem de máquina a seguir. Consideremos que cada célula de memória é composta por uma capacidade 1 byte.

```
DIM NOTA(12)
DATA 26.5, 45, 29
FOR I = 0 TO 12
READ NOTA(I)
NEXT I
```

E	C	E	C	E	C	E	C	E	C
70	01000100	7E	01000001	8C	00110010	9A	01001111	A8	00101000
71	01001001	7F	01010100	8D	00111001	9B	00100000	A9	01001001
72	01001101	80	01000001	8E	10001101	9C	00110001	AA	00101001
73	00100000	81	00100000	8F	01000110	9D	00110010	AB	10001101
74	01001110	82	00110010	90	01001111	9E	10001101	AC	01001110
75	01001111	83	00110110	91	01010010	9F	01010010	AD	01000101
76	01010100	84	00101110	92	00100000	A0	01000101	AE	01011000
77	01000001	85	00110101	93	01001001	A1	01000001	AF	01010100
78	00101000	86	00101100	94	00100000	A2	01000100	B0	00100000
79	00110001	87	00100000	95	00111101	A3	00100000	B1	01001001
7A	00110010	88	00110100	96	00100000	A4	01001110	B2	10001101
7B	00101001	89	00100101	97	00110000	A5	01001111		
7C	10001101	8A	00101100	98	00100000	A6	01010100		
7D	01000100	8B	00100000	99	01010100	A7	01000001		

E - Endereço da MP, em hexadecimal
C - Conteúdo da MP, célula por célula, em valores binários



5) Como a palavra **INFORMATICA** poderia estar codificada em ASCII?

6) Um algoritmo necessita colocar as três palavras em ordem crescente:

FEIJÃO		ARROZ
BATATA		BATATA
ARROZ		FEIJÃO

Como esse processo deve ser feito no computador, sendo que ele trabalha exclusivamente com correspondência por tabela?

Tipo Lógico

O tipo lógico permite a utilização de variáveis que possuam apenas dois valores para representação: **VERDADEIRO** representado pelo bit 1 e **FALSO** pelo bit 0.

Operações diversas podem ser utilizadas com esses dois valores. O processador opera esse dado pelas operações: AND, OR, NOT E XOR.



Operador lógico AND

O resultado da operação será VERDADE (1) **SE E SOMENTE SE** todas as variáveis forem VERDADE (1). Representada na literatura pelo símbolo ($\cdot$)

Tabela-verdade da Operação Lógica AND		
A	B	$X = A \text{ and } B \text{ ou } X = A \cdot B$
0	0	0
0	1	0
1	0	0
1	1	1

Operador lógico OR

O resultado da operação será VERDADE (1) **se pelo menos uma variável** for VERDADE (1). Representada na literatura pelo símbolo (+)

Tabela-verdade da Operação Lógica OR		
A	B	$X = A \text{ or } B \text{ ou } X = A + B$
0	0	0
0	1	1
1	0	1
1	1	1

Operador lógico NOT

Também conhecido como INVERSOR ou COMPLEMENTO, produz uma saída com um resultado oposto. Representado na literatura pelo símbolo (not A ou $\bar{A}$)

Tabela-verdade da Operação Lógica NOT	
A	$X = \text{NOT } A \text{ ou } X = \bar{A}$
0	1
1	0

Operador lógico XOR

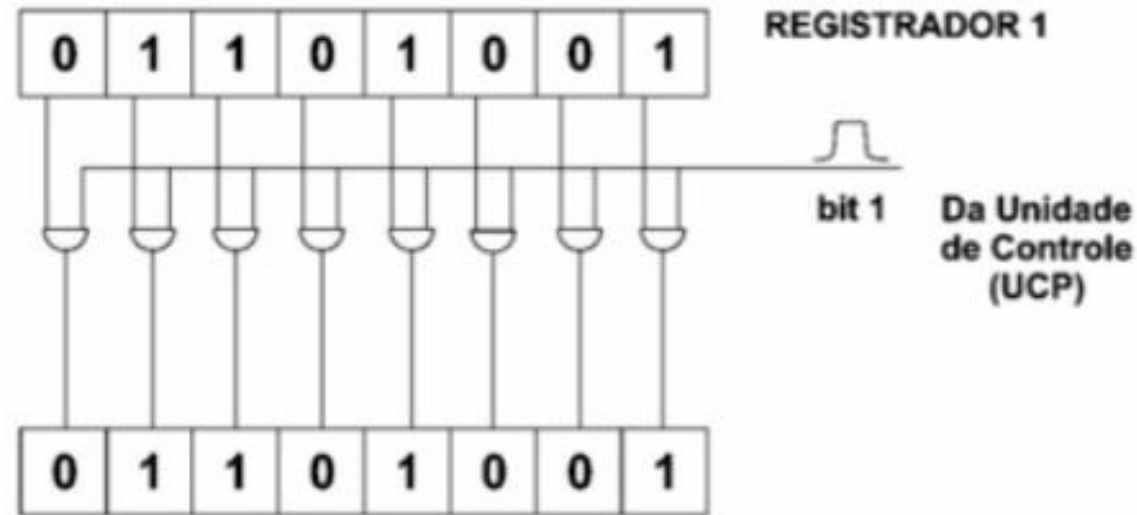
O resultado da operação será VERDADE (1) **se somente uma das variáveis** for VERDADE (1). Representada na literatura pelo símbolo ($\oplus$)

Tabela-verdade da Operação Lógica XOR		
A	B	$X = A \text{ xor } B$ ou $X = A \oplus B$
0	0	0
0	1	1
1	0	1
1	1	0

Exemplo de utilização da porta AND

O registrador 1 armazena o valor em binário 01101001. Deve-se transferir uma cópia deste valor para um registrador 2 ou para a memória.

Para isso, foi inserido um sinal de controle (bit 1) que é inserido na porta juntamente com o valor do registrador. A saída da porta AND irá para o registrador 2.



Exemplo de utilização da porta AND

O bit 1 combinado com o valor do registrador na porta AND, produz como resultado a cópia do valor. Pois:

$$0 \text{ AND } 1 = 0$$

$$1 \text{ AND } 1 = 1$$

$$1 \text{ AND } 1 = 1$$

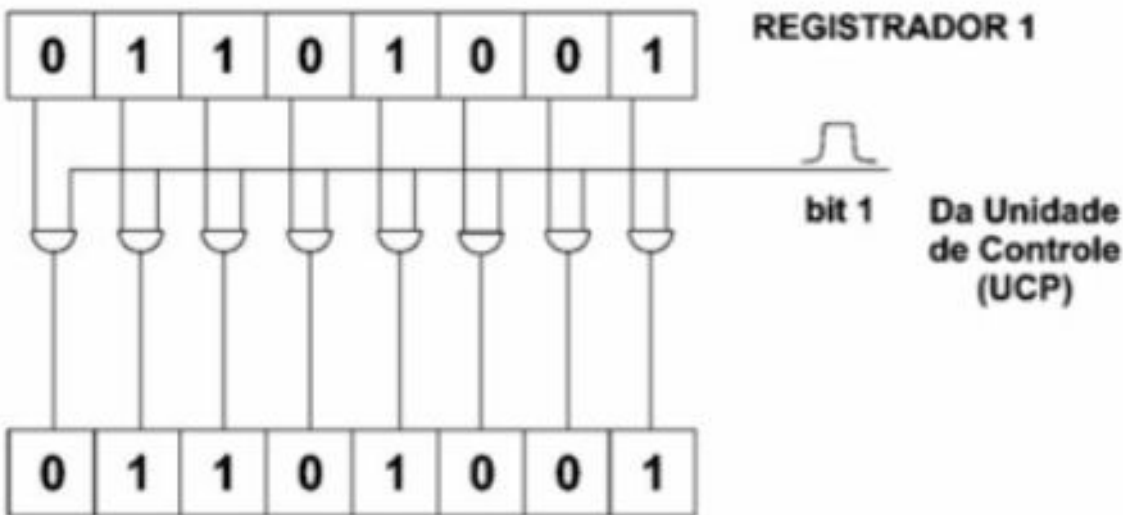
$$0 \text{ AND } 1 = 0$$

$$1 \text{ AND } 1 = 1$$

$$0 \text{ AND } 1 = 0$$

$$0 \text{ AND } 1 = 0$$

$$1 \text{ AND } 1 = 1$$





7) Considerando as variáveis A (1), B (0) e C (1) obtenha X:

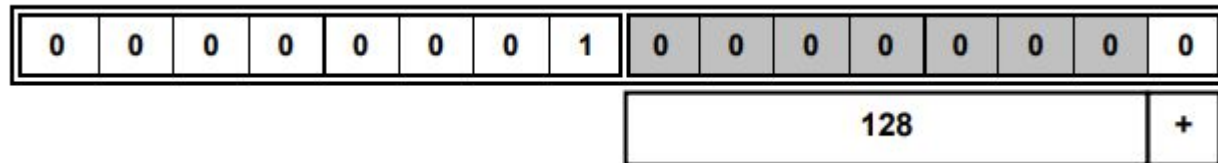
$$X = (\overline{A \oplus B}) + (C + \overline{A \cdot B}) \cdot \overline{C} \oplus A$$

Tipo Numérico

Para a representação de valores numéricos, os computadores utilizam um conjunto bits chamados de palavras (**word**).

O tamanho da palavra depende do computador utilizado e corresponde à quantidade de bits que este é capaz de transferir numa determinada operação.

Exemplo: o valor inteiro +128 definido em uma linguagem de programação, utiliza 2 bytes



Tipo Numérico

Porém, devemos pensar em algumas questões referentes ao tipo numérico:

- 1) Como um computador representa o sinal de um número?
- 2) Como um computador representaria uma vírgula em um número?
- 3) Já que um computador possui tamanho de memória limitado, o que aconteceria se um número excedesse a quantidade de memória, já que são infinitos?



Sinal e magnitude

Para representar o sinal de um número, utiliza-se um bit a mais na esquerda do número, como o **bit mais significativo**. Dessa forma:

Números positivos recebem o bit 0 (+)

Números negativos recebem o bit 1 (-)

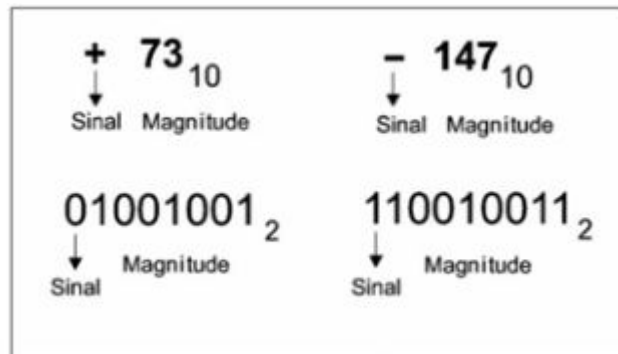
Valor decimal	Valor binário	
	Bit de sinal	
	↓	
+12	01100	(com 5 bits)
-12	11100	(com 5 bits)
-47	10010111	(com 9 bits)
+47	00010111	(com 9 bits)

Sinal e magnitude

Essa convenção é chamada de **sinal e magnitude**, onde há 1 bit à esquerda do número representando o sinal e os $n-1$ bits restantes que representam o número.



0000110110 = + 54
ou
1000110110 = - 54



Números inteiros

Para representar números inteiros, o computador pode utilizar os métodos de:

- **Sinal e magnitude;**
- **Complemento a 1;**
- **Complemento a 2.**

Complemento a 1

Para representar números inteiros positivos e negativos, o computador pode realizar o complemento a 1, onde:

Utiliza-se o bit mais à esquerda para o sinal, correspondendo o 0 ao sinal + e o 1 ao sinal -. Para os números positivos, os N-1 bits da direita representam o módulo. Para os negativos há complemento de todos os seus dígitos, ou seja: troca-se 0 por 1 e vice-versa. Inclui-se o bit de sinal.

Complemento a 1

Exemplo: Representar 10 e -10
Limitação de 8 bits ($N = 8$)

10	0	0001010
↓	↓	↓
nº	sinal	módulo

Número -10 é o complemento do seu simétrico

-10	1	1110101
↓	↓	↓
nº	sinal	módulo

Complemento a 1

Vantagens de utilizar o complemento a 1: possuir uma faixa simétrica e facilitar operações aritméticas.

Desvantagem: 2 representações para o número 0. Para 8 bits o 0 tem as seguintes representações:

00000000 (+0)

11111111 (-0)

Complemento a 1

Neste caso, a faixa de representação é:

$$- 2^{N-1} + 1 \leq X \leq + 2^{N-1} - 1$$

Para 8 bits a faixa é: $- 127 \leq X \leq +127$

Para 16 bits a faixa é: $- 32767 \leq X \leq + 32767$

Para 32 bits a faixa é: $- 2147483647 \leq X \leq + 2147483647$

Complemento a 2

Para o complemento a 2, utiliza-se o bit mais à esquerda para o sinal, e obtém-se o complemento de todos os bits do número positivo.

Depois disso, ao resultado obtido, soma-se 1 (em binário), desprezando o último transporte, se houver.

Complemento a 2

Número 10:

10	0	0001010
↓	↓	↓
nº	sinal	módulo

Número -10:

1º passo: complemento de 1

-10	1	1110101
↓	↓	↓
nº	sinal	módulo

2º passo:

1110101
+ 1

1110110

Complemento a 2

Vantagem: uma única representação para o 0 e facilidade nas operações aritméticas.

Para 8 bits teremos:

$$\begin{array}{r} 00000000 \text{ (+0)} \\ 11111111 \text{ (-0)} \\ \hline 1 \\ 10000000 \\ \swarrow \\ \text{estouro desprezado} \end{array}$$

Logo 0 e -0 têm a mesma representação.

Desvantagem: Assimetria. existem mais valores negativos que positivos.

Comparações

Atualmente, a maioria dos computadores para representar números utilizam o complemento 2. Isso se deve ao fato de haver questões mais vantajosas em termos de custo e eficiência entre os três métodos. A comparação está a seguir:

Tipo de representação	Dupla representação para o zero	Custo	Velocidade
Sinal e magnitude	SIM (desvantagem)	Alto (componentes separados para soma e subtração)	Baixa (algoritmo de verif. sinais, soma e subtração)
Complemento a 1	SIM (desvantagem)	Baixo (um componente único para soma e subtração)	Média (operação mais demorada que em C2)
Complemento a 2	NÃO (vantagem)	Baixo (um componente único para soma e subtração)	Alta (algoritmo simples e igual para soma e subtração)

Complemento a 2

Exercício



- 8) Como os números 20 e 23 ficam representados em sinal e magnitude?
- 9) Transforme os números abaixo em binário, utilizando complemento a 2:
- a) -7
 - b) -3
 - c) -45
 - d) -26

Números reais

Havia uma grande dificuldade para representar números com vírgula, pois em binário, não há símbolo que represente uma vírgula. Somente em texto livre essa representação pode ser feita. Exemplo da representação em ASCII:

+37,5 ₁₀	00101011	00110011	00110111	00101100	00110101
	+	3	7		5

Uma vírgula ou ponto separa a parte inteira da parte fracionária.

Para a representação, um computador “ASSUME” a sua posição em um número e esse número seria representado sem vírgula como se fosse um inteiro.

Por exemplo: o número 110111,110 seria representado por 110111110 e o sistema “saberia” que os três último dígitos à direita seriam fracionários.

Números reais

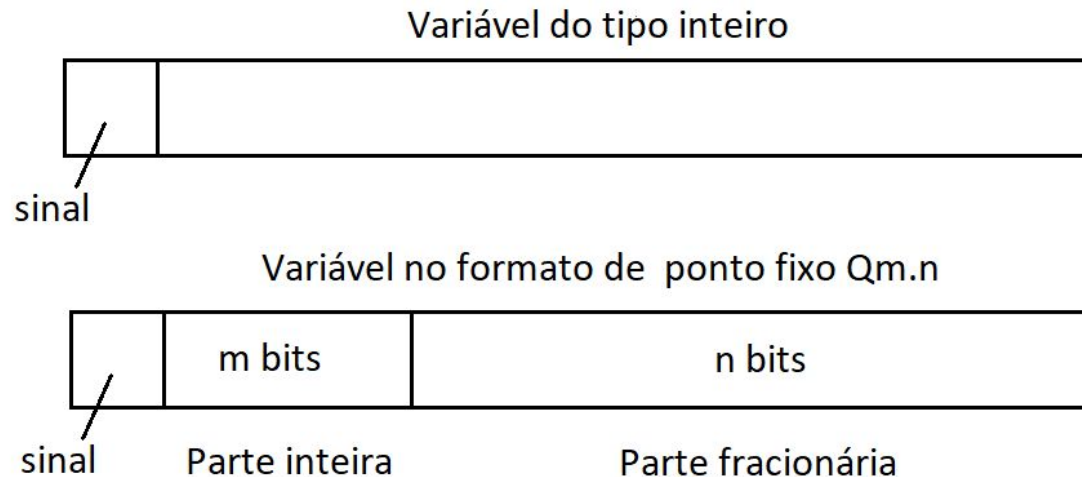
Para que um sistema “SAIBA” qual é a quantidade de algarismos inteiros e fracionários, utiliza-se duas formas de representação:

representação em ponto fixo

representação em ponto flutuante

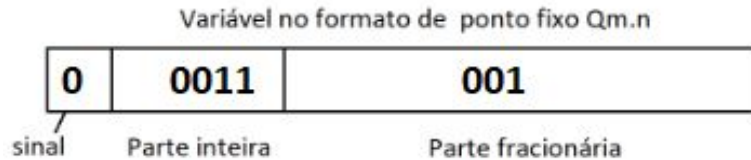
Representação em ponto fixo

Para a representação em ponto fixo, há uma quantidade de bits disponíveis para representar o sinal, a parte inteira e a parte fracionária.

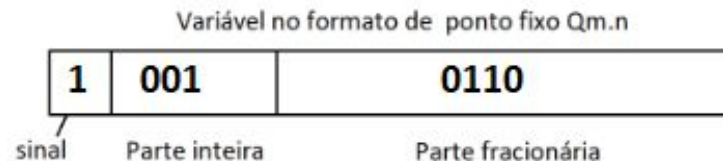


Representação em ponto fixo

Exemplo: +11,001 (em um registrador de 8 bits, sendo a parte inteira representada por 4 bits e fracionária por 3).



Exemplo: - 1,0110 (em um registrador de 8 bits, sendo a parte inteira representada por 3 bits e fracionária por 4).

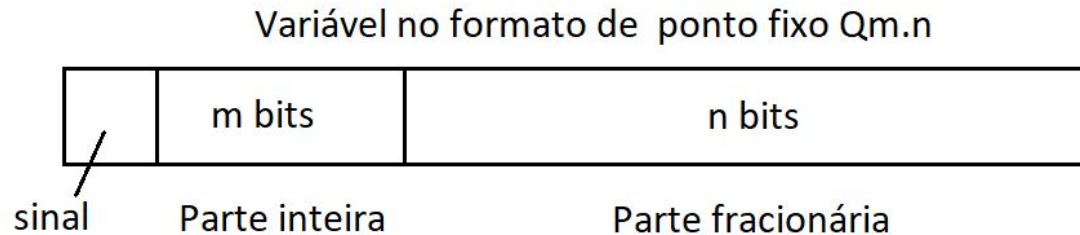


Representação em ponto fixo

Exemplo: como o número $11111.111_{(2)} = 31,875_{(10)}$ pode ser representado em ponto fixo para um registrador de 8 bits?

$$(31,875 \text{ pois: } 2^4 + 2^3 + 2^2 + 2^1 + 2^0 + 2^{-1} + 2^{-2} + 2^{-3})$$

OBS: Se o sistema é de 8 bits, onde coloca-se o sinal?



Representação em ponto fixo

Uma questão a ser observada na representação por ponto fixo é que apesar de um número ter um tamanho ilimitado, um computador precisa de tempo para a computação desses números e possui uma capacidade de memória limitada.

Por conta disso, um número poder ser:

Truncado: todos os números além da precisão são descartados;

Arredondado: o próximo dígito da representação é incrementado.

A representação em ponto fixo tem uma precisão pré-definida.

Representação em ponto fixo

Exemplo: considerando uma precisão de 3 dígitos:

1. $(0.122344)_5$

Truncamento: $(0.122)_5$ Arredondamento: $(0.123)_5$

2. 0.4593999

Truncamento: 0.459 Arredondamento: 0.459

3. 0.4595999

Truncamento: 0.459 Arredondamento: 0.460

Representação em ponto flutuante

Para driblar a dificuldade do ponto fixo em representar números muito grandes e exatos há a representação por ponto flutuante. A definição de ponto flutuante é:

O ponto varia de sua posição. Modificando em sequência a sua representação. (Flutua)
A representação pode variar ("flutuar") a posição da vírgula, ajustando a potência da base.
"FLOAT"

Qualquer número pode ser representado por uma notação científica.
Exemplo:

- $45,724 \times 10^0$
- 45724×10^{-3}
- $0,45724 \times 10^2$

Representação em ponto flutuante

Seria impossível representar em ponto fixo (em um registrador de 8 bits por exemplo)

- a) A massa da terra: 5.972.000.000.000.000.000.000
- b) O valor exato de PI: 3,14159265358979
- c) A distância em metros entre a terra e o sol: 149.600.000.000m

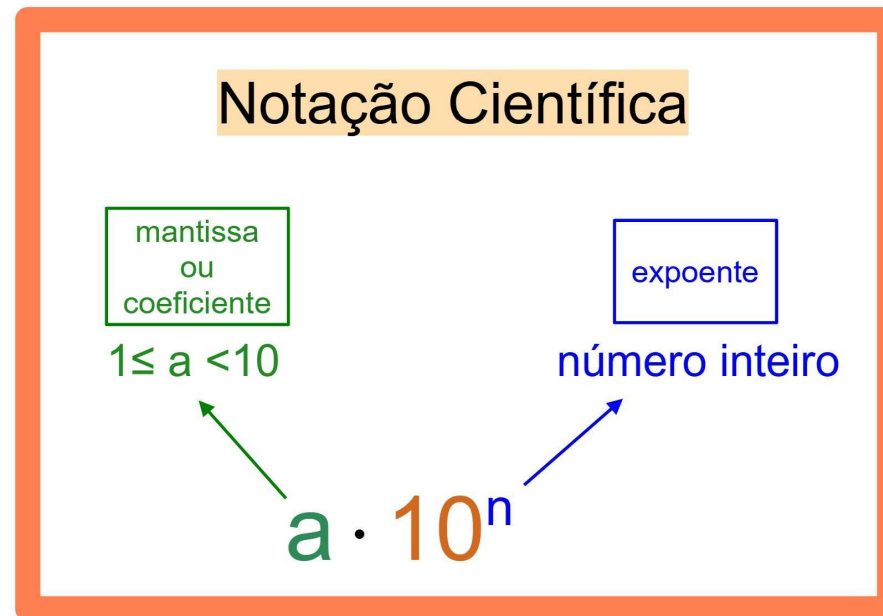


E se esses valores pudessem ser representados por notação científica?

- a) A massa da terra: $5,972 \times 10^{21}$ kg

Representação em ponto flutuante

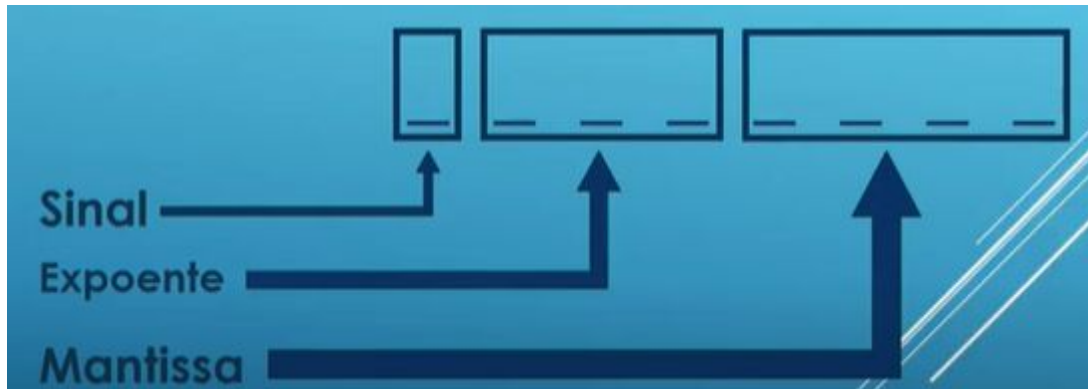
Uma notação científica é composta por:



Representação em ponto flutuante

Dessa forma, em uma unidade de registrador:

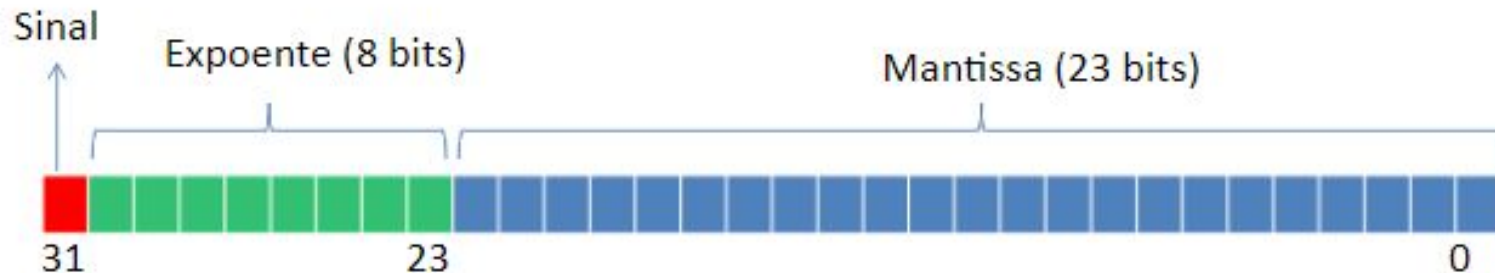
- 1) é reservado um bit para o sinal;
- 2) é reservada uma quantidade de bits para o expoente.
- 3) é reservada uma quantidade de bits para a mantissa (coeficiente);



Em um registrador de 8 bits por exemplo, 1 bit é para um sinal, 3 para o expoente e 4 para a mantissa. A quantidade é definida pelo fabricante.

Representação em ponto flutuante

O padrão IEEE 754-1985 regulamenta que a representação do ponto flutuante para 32 bits deve ser feita da seguinte forma:



1 bit para o sinal;
8 bits para o expoente;
23 bits para a mantissa.

Representação em ponto flutuante

Considerando o número $25,625_{(10)}$, em base 2 temos:

Parte inteira: $25_{(10)} = 11001_{(2)}$;

Parte fracionária: $0.625_{(10)} = .101_{(2)}$;

Ou seja:

11001 . 101

Representação em ponto flutuante

Deve-se mover o ponto até o 1º dígito não nulo e transformar em uma notação científica (forma normalizada - IEEE 754):

$$11001.101 \times 2^0;$$

$$1100.1101 \times 2^1;$$

$$110.01101 \times 2^2;$$

$$11.001101 \times 2^3;$$

$$1.1001101 \times 2^4;$$

Ou seja:

$$1.1001101 \times 2^4$$

Representação em ponto flutuante

Na base 2 o ÚNICO dígito não nulo é 1. Portanto, apaga-se o primeiro número (1) e vírgula:

$$\cancel{1}.1001101 \times 2^4$$

Ou seja:

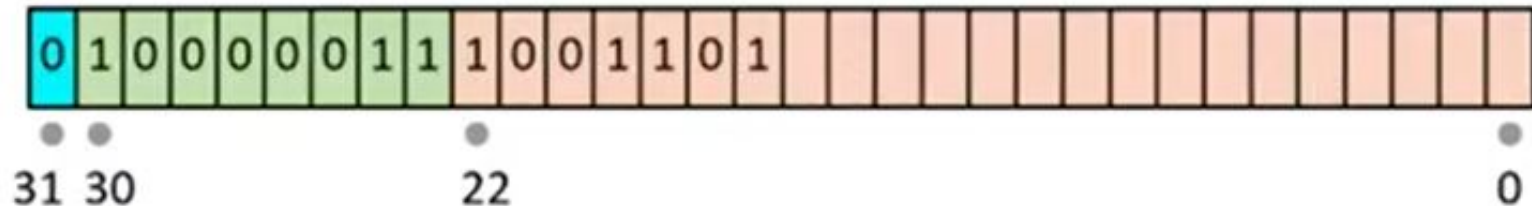
$$1001101 \times 2^4$$

O primeiro número não é armazenado. Só o que vem depois dele.

Representação em ponto flutuante

Diante disso:

- 1) O sinal é positivo, portanto 0;
- 2) O expoente é $4 + 127$ (o expoente é polarizado, pois como o primeiro número foi excluído **<2⁷> - 1**, deve-se compensar). Portanto: $4 + 127 = 131$ que em binário significa $10000011_{(2)}$;
- 3) A mantissa é o coeficiente resultante, sendo preenchida com os demais espaços por 0 até completar o espaço de 23 bits : $1001101 + 23$ zeros.



Representação de Infinito

+ Infinito

Sinal igual a 0

Todo o expoente é igual a 1

Toda a mantissa é igual a 0

- Infinito

Sinal igual a 1

Todo o expoente é igual a 1

Toda a mantissa é igual a 0

+infinito (utilizando 32 bits – *single*)

0 1 1 1 1 1 1 1 0

-infinito (utilizando 32 bits – *single*)

1 1 1 1 1 1 1 1 0

Representação de NaN

Caso ocorra um erro ocorre **NaN: NOT A NUMBER**. Dessa forma:

O bit de sinal é igual a 0;

Todos os bits do expoente são iguais a 1

A mantissa deverá ser um valor qualquer diferente de zero



Representação de número 0

Para representar o número 0:

O bit de sinal é igual a 0 ou 1;

Todos os bits do expoente são iguais a 0

Todos os bits da mantissa são iguais a 0

+0 (utilizando 32 bits – *single*)

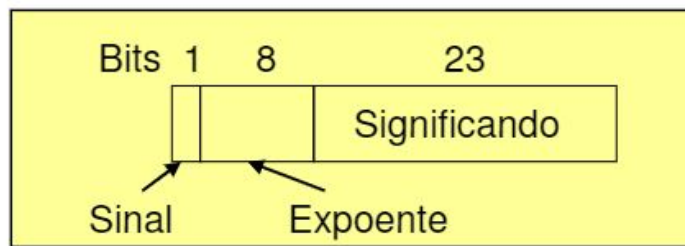


-0 (utilizando 32 bits – *single*)

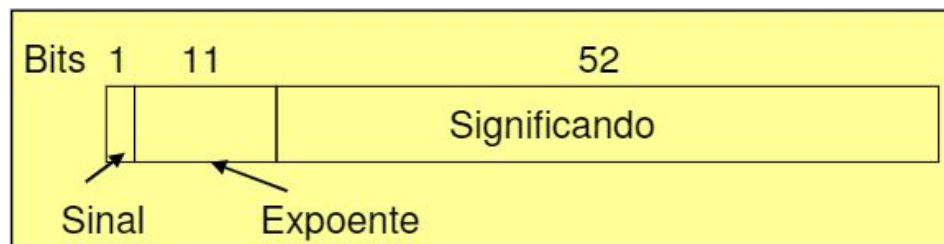


Representação em ponto flutuante

O padrão IEE 754 definiu a representação de ponto flutuante por precisão simples (float - 32 bits), dupla (double - 64 bits) e estendida por (80 bits).



Precisão simples



Precisão dupla



10) Realize as conversões abaixo:

a) $10,875_{(10)} = (\quad ? \quad)$ (IEEE 754, com 32 bits)

b) $11000001110100000000000000000000_{(2)} = (\quad ? \quad)$ (IEEE 754, com 32 bits)

c) $2,64_{(10)} = (\quad ? \quad)$ (IEEE 754, com 32 bits)

d) $5,5_{(10)} = (\quad ? \quad)$ (IEEE 754, com 32 bits)

e) $-2,625_{(2)} = (\quad ? \quad)$ (IEEE 754, com 32 bits)

Representação em ponto flutuante

Exercício



f) $11/2_{(10)} = (\quad ? \quad)$ (IEEE 754, com 32 bits)

g) $-3,125_{(10)} = (\quad ? \quad)$ (IEEE 754, com 32 bits)

h) $45,3_{(10)} = (\quad ? \quad)$ (IEEE 754, com 32 bits)

i) $17/3_{(10)} = (\quad ? \quad)$ (IEEE 754, com 32 bits)

Overflow e Underflow

Quando o valor de um número n excede a capacidade de memória de computador ($n-1$) ocorre o que chamamos de *overflow*.

Se o valor de um dado a ser representado excede a capacidade de representação, ocorre *overflow*, pois seria insuficiente para representar o dado.

Geralmente ocorre *overflow* em operações de multiplicação. Vide calculadora.

O sistema operacional é responsável por tomar providências quando o *overflow* ocorre.

Já o *underflow* ocorre quando o valor absoluto do dado a ser representado é tão pequeno que fica menor que o menor valor absoluto representável.

Overflow e Underflow

Por conta de *overflow*, alguns valores em ponto flutuante podem perder a precisão.

Para representação de números em que é necessário manter precisão até o último algarismo, não é admissível erro por aproximação.

Para evitar isso, utiliza-se a correspondência de cada número por tabela BCD.

A ideia do BCD é representar, em binário, cada algarismo de forma que o número original seja integralmente preservado.

Overflow e Underflow

Exemplo: $239_{10} = (?) \text{BCD}$

▪ $2 = 0010_2$

▪ $3 = 0011_2$ e

▪ $9 = 1001_2$, logo: **$239 = 001000111001 \text{ (BCD)}$** .

Overflow e Underflow

Exercício



- 11) Um registrador de 8 bits receberá multiplicação de 111011×111 . Neste caso, poderá ocorrer *overflow*?
- 12) Um registrador de 8 bits receberá multiplicação de 10011×11 . Neste caso, poderá ocorrer *overflow*?
- 13) Um registrador de 5 bits receberá a soma de $1111 + 1101$. Neste caso, poderá ocorrer underflow?
- 14) Um registrador de 16 bits receberá multiplicação de 111011110101×11 . Neste caso, poderá ocorrer overflow?

MONTEIRO, Mário A. **Organização de Computadores**. Ed. LTC, 5ª edição, 2015.